

消息队列面试题

 面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

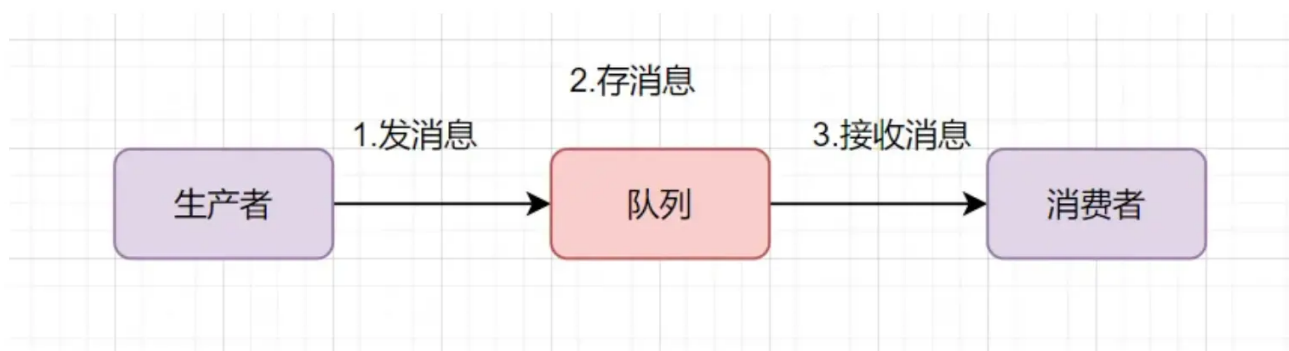
刷高频题拿高薪offer



消息队列场景

什么是消息队列？

你可以把消息队列理解为一个**使用队列来通信**的组件。它的本质，就是个**转发器**，包含**发消息**、**存消息**、**消费消息**的过程。最简单的消息队列模型如下：



我们通常说的消息队列，简称**MQ (Message Queue)**，它其实就指**消息中间件**，当前业界比较流行的开源消息中间件包括：RabbitMQ、RocketMQ、Kafka。

消息队列怎么选型？

Kafka、ActiveMQ、RabbitMQ、RocketMQ来进行不同维度对比。

特性	ActiveMQ	RabbitMQ	RocketMQ	Kafka
单机吞吐量	万级	万级	10 万级	10 万级
时效性	毫秒级	微秒级	毫秒级	毫秒级
可用性	高（主从）	高（主从）	非常高（分布式）	非常高（分布式）
消息重复	至少一次	至少一次	至少一次 最多一次	至少一次最多一次
消息顺序性	有序	有序	有序	分区有序
支持主题数	千级	百万级	千级	百级，多了性能严重下滑
消息回溯	不支持	不支持	支持（按时间回溯）	支持（按offset回溯）
管理界面	普通	普通	完善	普通

选型的时候，我们需要根据业务场景，结合上述特性来进行选型。

比如你要支持天猫双十一类超大型的秒杀活动，这种一锤子买卖，那管理界面、消息回溯啥的不重要。

我们需要看什么？看吞吐量！

所以优先选Kafka和RocketMQ这种更高吞吐的。

比如做一个公司的中台，对外提供能力，那可能会有很多主题接入，这时候主题个数又是很重要的考量，像Kafka这样百级的，就不太符合要求，可以根据情况考虑千级的RocketMQ，甚至百万级的RabbitMQ。

又比如是一个金融类业务，那么重点考虑的就是稳定性、安全性，分布式部署的Kafka和Rocket就更有优势。

特别说一下时效性，RabbitMQ以微秒的时效作为招牌，但实际上毫秒和微秒，在绝大多数情况下，都没有感知的区别，加上网络带来的波动，这一点在生产过程中，反而不会作为重要的考量。

其它的特性，如消息确认、消息回溯，也经常作为考量的场景，管理界面的话试公司而定了，反正我呆过的地方，都不看重这个，毕竟都有自己的运维体系。

消息队列使用场景有哪些？

- **解耦**：可以在多个系统之间进行解耦，将原本通过网络之间的调用的方式改为使用MQ进行消息的异步通讯，只要该操作不是需要同步的，就可以改为使用MQ进行不同系统之间的联系，这样项目之间不会存在耦合，系统之间不会产生太大的影响，就算一个系统挂了，也只是消息挤压在MQ里面没人进行消费而已，不会对其他的系统产生影响。
- **异步**：加入一个操作设计到好几个步骤，这些步骤之间不需要同步完成，比如客户去创建了一个订单，还要去客户轨迹系统添加一条轨迹、去库存系统更新库存、去客户系统修改客户的状态等等。这样如果这个系统都直接进行调用，那么将会产生大量的时间，这样对于客户是无法接收的；并且像添加客户轨迹这种操作是不需要去同步操作的，如果使用MQ将客户创建订单时，将后面的轨迹、库存、状态等信息的更新全都放到MQ里面然后去异步操作，这样就可加快系统的访问速度，提供更好的客户体验。
- **削峰**：一个系统访问流量有高峰时期，也有低峰时期，比如说，中午整点有一个抢购活动等等。比如系统平时流量并不高，一秒钟只有100多个并发请求，系统处理没有任何压力，一切风平浪静，到了某个抢购活动时间，系统并发访问了剧增，比如达到了每秒5000个并发请求，而我们的系统每秒只能处理2000个请求，那么由于流量太大，我们的系统、数据库可能就会崩溃。这时如果使用MQ进行流量削峰，将用户的大量消息直接

放到MQ里面，然后我们的系统去按自己的最大消费能力去消费这些消息，就可以保证系统的稳定，只是可能要跟进业务逻辑，给用户返回特定页面或者稍后通过其他方式通知其结果

消息重复消费怎么解决？

生产端为了保证消息发送成功，可能会重复推送(直到收到成功ACK)，会产生重复消息。但是一个成熟的MQ Server框架一般会想办法解决，避免存储重复消息(比如：空间换时间，存储已处理过的message_id)，给生产端提供一个幂等性的发送消息接口。

但是消费端却无法根本解决这个问题，在高并发标准要求下，拉取消息+业务处理+提交消费位移需要做事务处理，另外消费端服务可能宕机，很可能会拉取到重复消息。

所以，只能业务端自己做控制，对于已经消费成功的消息，本地数据库表或Redis缓存业务标识，每次处理前先进行校验，保证幂等。

消息丢失怎么解决的？

使用一个消息队列，其实就分为三大块：**生产者、中间件、消费者**，所以要保证消息就是保证三个环节都不能丢失数据。



- **消息生产阶段**：生产者会不会丢消息，取决于生产者对于异常情况的处理是否合理。从消息被生产出来，然后提交给 MQ 的过程中，只要能正常收到（MQ 中间件）的 ack 确认响应，就表示发送成功，所以只要处理好返回值和异常，如果返回异常则进行消息重发，那么这个阶段是不会出现消息丢失的。
- **消息存储阶段**：Kafka 在使用时是部署一个集群，生产者在发布消息时，队列中间件通常会写「多个节点」，也就是有多个副本，这样一来，即便其中一个节点挂了，也能保证集群的数据不丢失。
- **消息消费阶段**：消费者接收消息+消息处理之后，才回复 ack 的话，那么消息阶段的消息不会丢失。不能收到消息就回 ack，否则可能消息处理中途挂掉了，消息就丢失了。

使用消息队列还应该注意哪些问题？

需要考虑消息可靠性和顺序性方面的问题。

消息队列的可靠性、顺序性怎么保证？

消息可靠性可以通过下面这些方式来保证

- **消息持久化**：确保消息队列能够持久化消息是非常关键的。在系统崩溃、重启或者网络故障等情况下，未处理的消息不应丢失。例如，像 RabbitMQ 可以通过配置将消息持久化到磁盘，通过将队列和消息都设置为持久化的方式（设置 `durable = true`），这样在服务器重启后，消息依然可以被重新读取和处理。
- **消息确认机制**：消费者在成功处理消息后，应该向消息队列发送确认（acknowledgment）。消息队列只有收到确认后，才会将消息从队列中移除。如果没有收到确认，消息队列可能会在一定时间后重新发送消息给

其他消费者或者再次发送给同一个消费者。以 Kafka 为例，消费者通过 `commitSync` 或者 `commitAsync` 方法来提交偏移量（offset），从而确认消息的消费。

- **消息重试策略**：当消费者处理消息失败时，需要有合理重试策略。可以设置重试次数和重试间隔时间。例如，在第一次处理失败后，等待一段时间（如 5 秒）后进行第二次重试，如果重试多次（如 3 次）后仍然失败，可以将消息发送到死信队列，以便后续人工排查或者采取其他特殊处理。

消息顺序性保证的方式如下：

- **有序消息处理场景识别**：首先需要明确业务场景中哪些消息是需要保证顺序的。例如，在金融交易系统中，对于同用户的转账操作顺序是不能打乱的。对于需要顺序处理的消息，要确保消息队列和消费者能够按照特定的顺序进行处理。
- **消息队列对顺序性的支持**：部分消息队列本身提供了顺序性保证的功能。比如 Kafka 可以通过将消息划分到同一个分区（Partition）来保证消息在分区内是有序的，消费者按照分区顺序读取消息就可以保证消息顺序。但这也可能会限制消息的并行处理程度，需要在顺序性和吞吐量之间进行权衡。
- **消费者顺序处理策略**：消费者在处理顺序消息时，应该避免并发处理可能导致顺序打乱的情况。例如，可以通过单线程或者使用线程池并对顺序消息进行串行化处理等方式，确保消息按照正确的顺序被消费。

如何保证幂等写？

幂等性是指 **同一操作的多次执行对系统状态的影响与一次执行结果一致**。例如，支付接口若因网络重试被多次调用，最终应确保仅扣款一次。实现幂等写的核心方案：

- **唯一标识（幂等键）**：客户端为每个请求生成全局唯一ID（如 UUID、业务主键），服务端校验该ID是否已处理，适用场景接口调用、消息消费等。
- **数据库事务 + 乐观锁**：通过版本号或状态字段控制并发更新，确保多次更新等同于单次操作，适用场景数据库记录更新（如余额扣减、订单状态变更）。
- **数据库唯一约束**：利用数据库唯一索引防止重复数据写入，适用场景数据插入场景（如订单创建）。
- **分布式锁**：通过锁机制保证同一时刻仅有一个请求执行关键操作，适用场景高并发下的资源抢夺（如秒杀）。
- **消息去重**：消息队列生产者每条消息生成唯一的消息 ID，消费者在处理消息前，先检查该消息 ID 是否已经处理过，如果已经处理过则丢弃该消息。

如何处理消息队列的消息积压问题？

消息积压是因为生产者的生产速度，大于消费者的消费速度。遇到消息积压问题时，我们需要先排查，是不是有 bug 产生了。

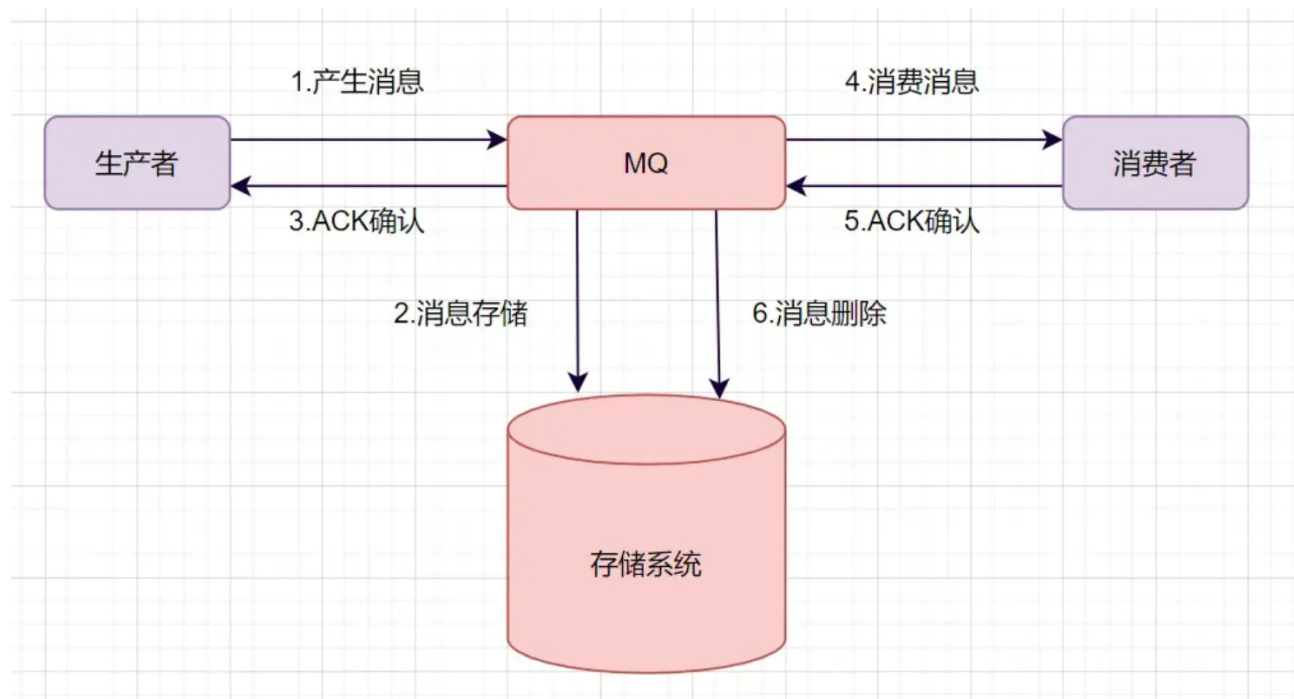
如果不是 bug，我们可以 **优化一下消费的逻辑**，比如之前是一条一条消息消费处理的话，我们可以确认是不是可以优化为 **批量处理消息**。如果还是慢，我们可以考虑水平扩容，增加 Topic 的队列数，和消费组机器的数量，提升整体消费能力。

如果是 bug 导致几百万消息持续积压几小时。有如何处理呢？需要解决 bug，**临时紧急扩容**，大概思路如下：

1. 先修复 consumer 消费者的问题，以确保其恢复消费速度，然后将现有 consumer 都停掉。
2. 新建一个 topic，partition 是原来的 10 倍，临时建立好原先 10 倍的 queue 数量。
3. 然后写一个临时的分发数据的 consumer 程序，这个程序部署上去消费积压的数据，消费之后不做耗时的处理，直接均匀轮询写入临时建立好的 10 倍数量的 queue。
4. 接着临时征用 10 倍的机器来部署 consumer，每一批 consumer 消费一个临时 queue 的数据。这种做法相当于是临时将 queue 资源和 consumer 资源扩大 10 倍，以正常的 10 倍速度来消费数据。
5. 等快速消费完积压数据之后，得恢复原先部署的架构，重新用原先的 consumer 机器来消费消息。

如何保证数据一致性，事务消息如何实现？

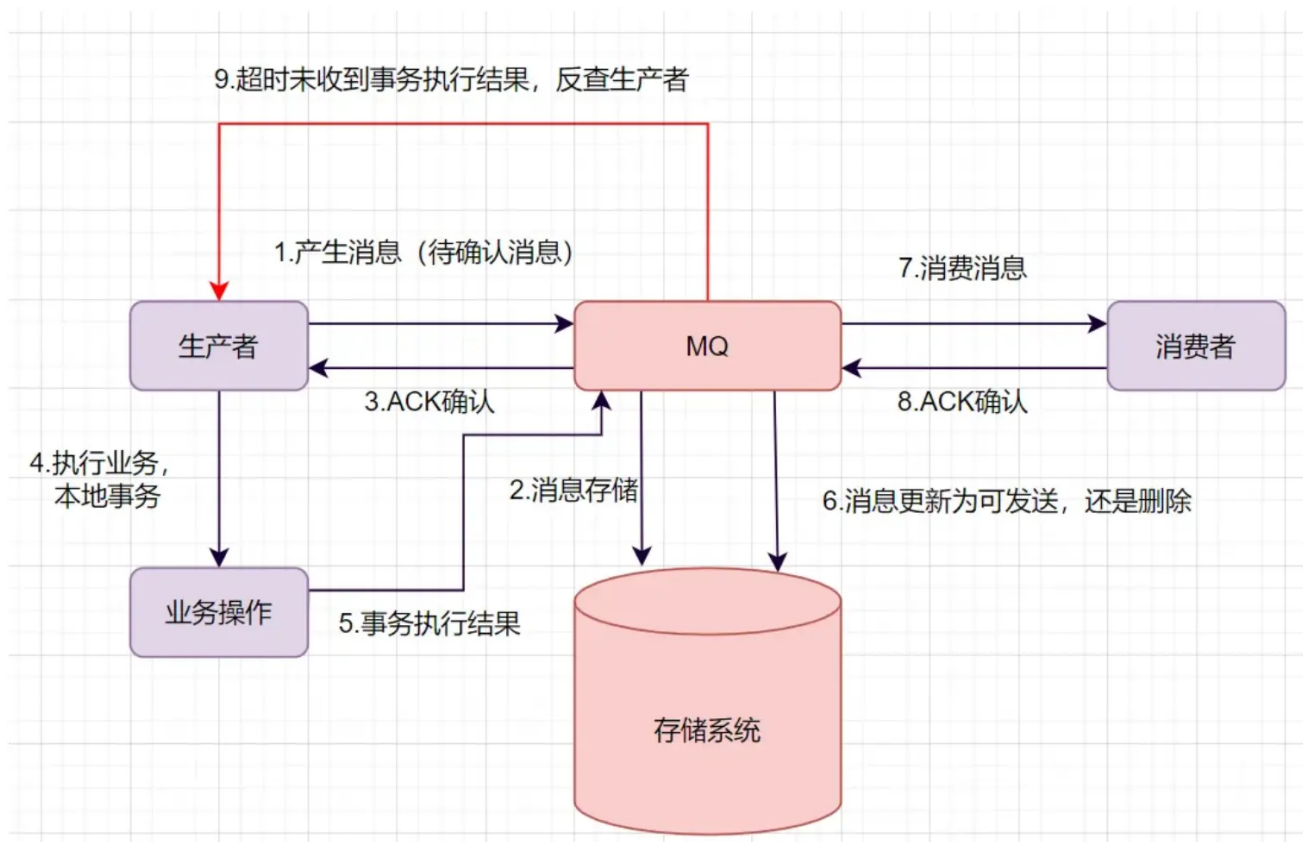
一条普通的MQ消息，从产生到被消费，大概流程如下：



1. 生产者产生消息，发送带MQ服务器
2. MQ收到消息后，将消息持久化到存储系统。
3. MQ服务器返回ACK到生产者。
4. MQ服务器把消息push给消费者
5. 消费者消费完消息，响应ACK
6. MQ服务器收到ACK，认为消息消费成功，即在存储中删除消息。

我们举个**下订单**的例子吧。订单系统创建完订单后，再发送消息给下游系统。如果订单创建成功，然后消息没有成功发送出去，下游系统就无法感知这个事情，出导致数据不一致。

如何保证数据一致性呢？可以使用**事务消息**。一起来看下事务消息是如何实现的吧。



1. 生产者产生消息，发送一条**半事务消息**到MQ服务器
2. MQ收到消息后，将消息持久化到存储系统，这条消息的状态是**待发送**状态。
3. MQ服务器返回ACK确认到生产者，此时MQ不会触发消息推送事件
4. 生产者执行本地事务
5. 如果本地事务执行成功，即commit执行结果到MQ服务器；如果执行失败，发送rollback。
6. 如果是正常的commit，MQ服务器更新消息状态为**可发送**；如果是rollback，即删除消息。
7. 如果消息状态更新为可发送，则MQ服务器会push消息给消费者。消费者消费完就回ACK。
8. 如果MQ服务器长时间没有收到生产者的commit或者rollback，它会反查生产者，然后根据查询到的结果执行最终状态。

消息队列是参考哪种设计模式？

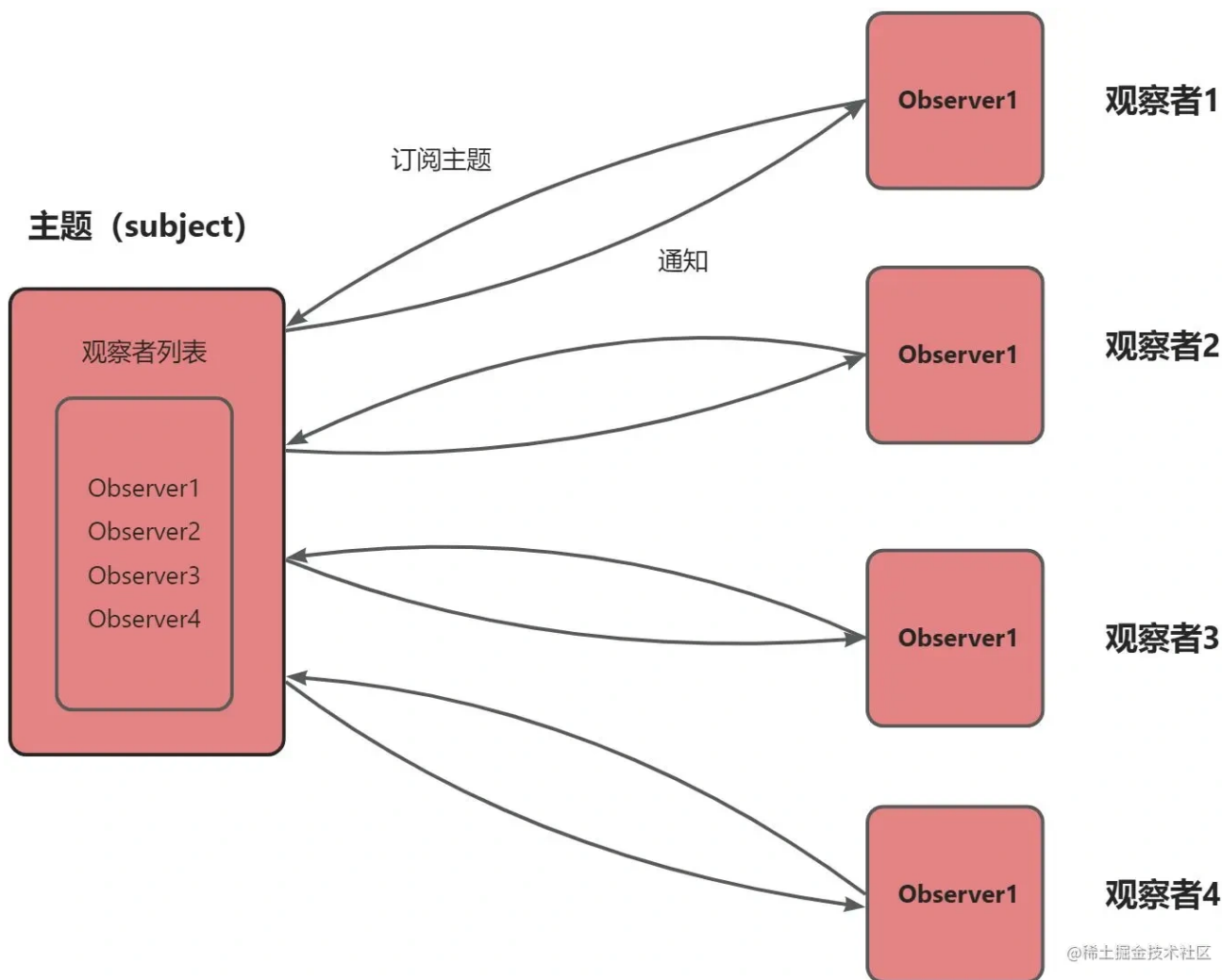
是参考了观察者模式和发布订阅模式，两种设计模式思路是一样的，举个生活例子：

- 观察者模式：某公司给自己员工发月饼发粽子，是由公司的行政部门发送的，这件事不适合交给第三方，原因是“公司”和“员工”是一个整体
- 发布-订阅模式：某公司要给别人发各种快递，因为“公司”和“其他人”是独立的，其唯一的桥梁是“快递”，所以这件事适合交给第三方快递公司解决

上述过程中，如果公司自己去管理快递的配送，那公司就会变成一个快递公司，业务繁杂难以管理，影响公司自身的主营业务，因此使用何种模式需要考虑什么情况两者是需要耦合的

观察者模式

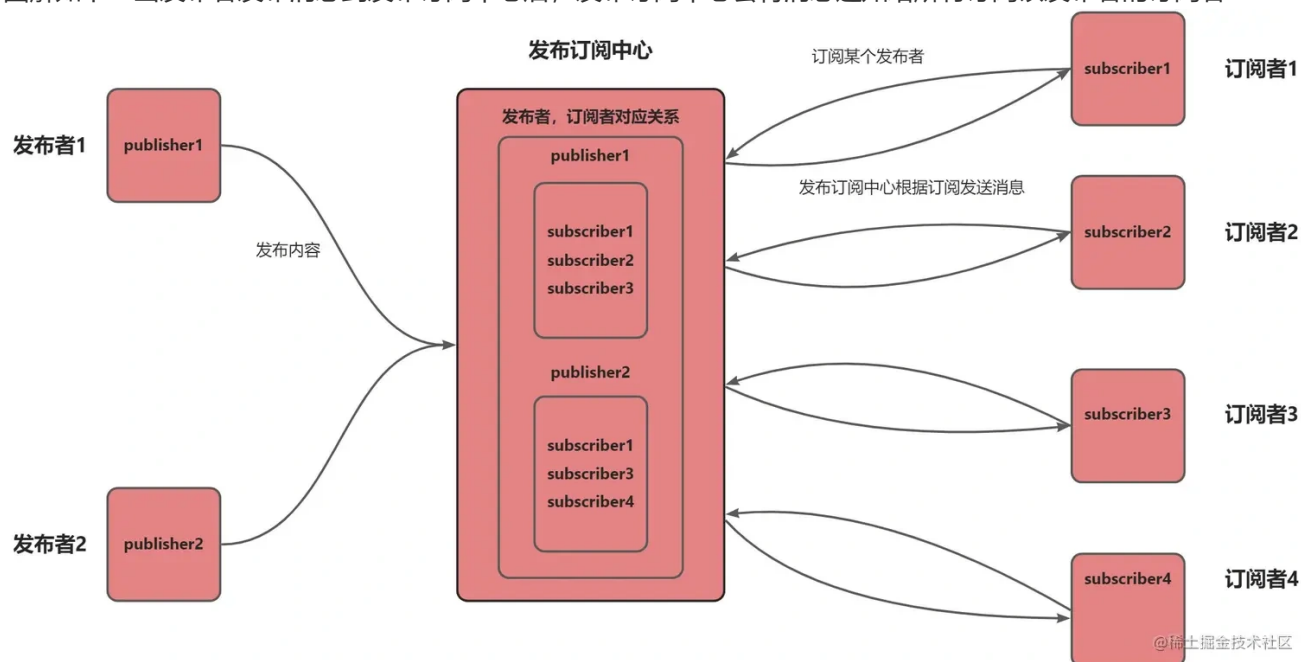
观察者模式实际上就是一个一对多的关系，在观察者模式中存在一个主题和多个观察者，主题也是被观察者，当我们主题发布消息时，会通知各个观察者，观察者将会收到最新消息，图解如下：每个观察者首先订阅主题，订阅成功后当主题发送消息时会循环整个观察者列表，逐一发送消息通知。



发布订阅模式

发布订阅模式和观察者模式的区别就是发布者和订阅者完全解耦，通过中间的发布订阅中心进行消息通知，发布者并不知道自己发布的消息会通知给谁，因此发布订阅模式有三个重要角色，发布者->发布订阅中心->订阅者。

图解如下：当发布者发布消息到发布订阅中心后，发布订阅中心会将消息通知给所有订阅该发布者的订阅者



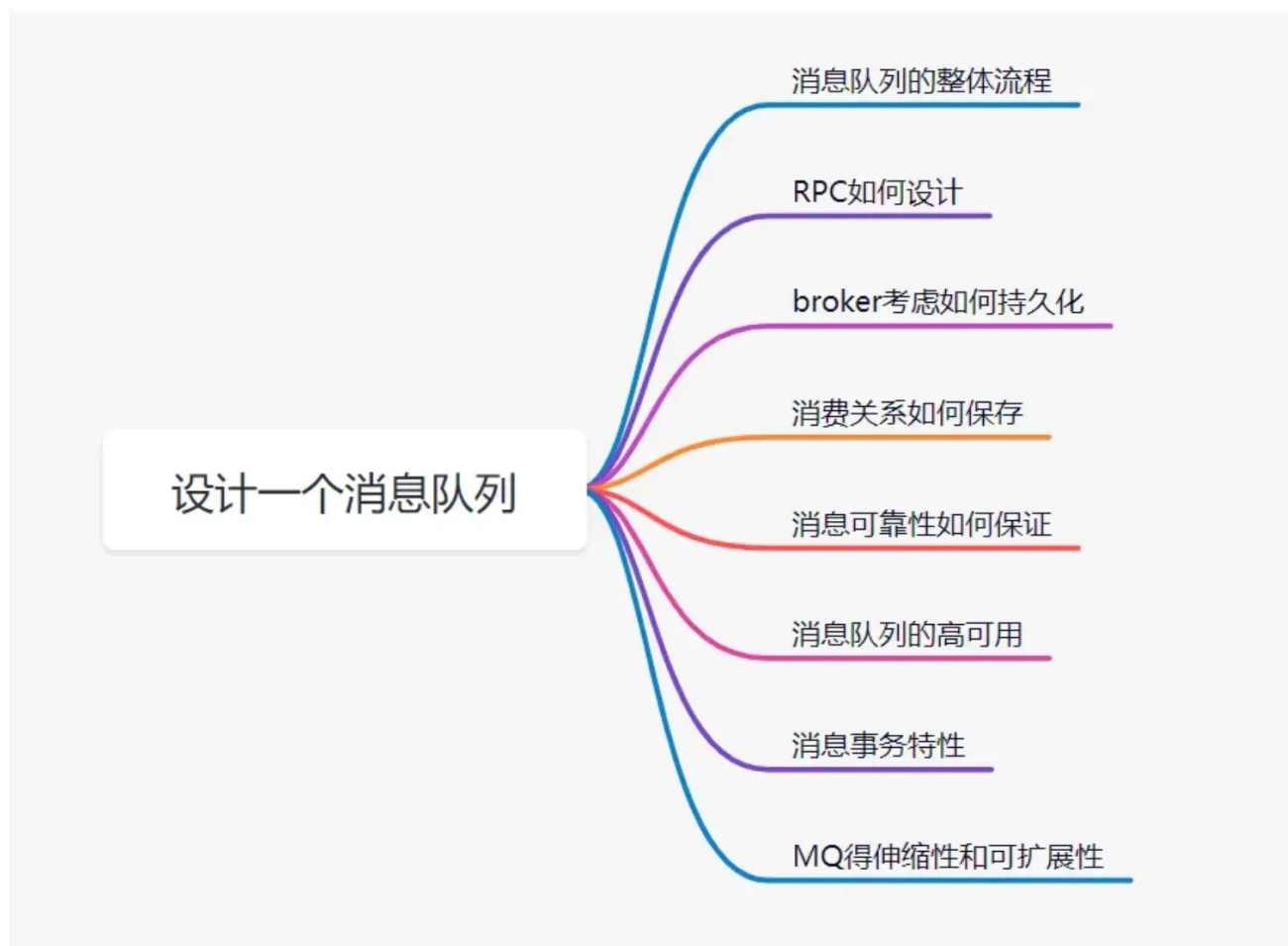
让你写一个消息队列，该如何进行架构设计？

这个问题面试官主要考察三个方面的知识：

- 你有没有对消息队列的架构原理比较了解
- 考察你的个人设计能力
- 考察编程思想，如什么高可用、可扩展性、幂等等等。

遇到这种设计题，大部分人会很蒙圈，因为平时没有思考过类似的问题。大多数人平时埋头增删改啥，不去思考框架背后的一些原理。有很多类似的问题，比如让你来设计一个 Dubbo 框架，或者让你来设计一个 MyBatis 框架，你会怎么思考呢？

回答这类问题，并不要求你研究过那技术的源码，你知道那个技术框架的基本结构、工作原理即可。设计一个消息队列，我们可以从这几个角度去思考：



1. 首先是消息队列的整体流程，producer发送消息给broker，broker存储好，broker再发送给consumer消费，consumer回复消费确认等。
2. producer发送消息给broker，broker发消息给consumer消费，那就需要两次RPC了，RPC如何设计呢？可以参考开源框架Dubbo，你可以说说服务发现、序列化协议等等
3. broker考虑如何持久化呢，是放文件系统还是数据库呢，会不会消息堆积呢，消息堆积如何处理呢。
4. 消费关系如何保存呢？点对点还是广播方式呢？广播关系又是如何维护呢？zk还是config server
5. 消息可靠性如何保证呢？如果消息重复了，如何幂等处理呢？
6. 消息队列的高可用如何设计呢？可以参考Kafka的高可用保障机制。多副本 -> leader & follower -> broker 挂了重新选举 leader 即可对外服务。

7. 消息事务特性，与本地业务同个事务，本地消息落库;消息投递到服务端，本地才删除；定时任务扫描本地消息库，补偿发送。
8. MQ得伸缩性和可扩展性，如果消息积压或者资源不够时，如何支持快速扩容，提高吞吐？可以参照一下Kafka的设计理念，broker -> topic -> partition，每个 partition 放一个机器，就存一部分数据。如果现在资源不够了，简单啊，给 topic 增加 partition，然后做数据迁移，增加机器，不就可以存放更多数据，提供更高的吞吐量了。

RocketMQ

消息队列为什么选择RocketMQ的？

项目用的是 RocketMQ 消息队列。选择RocketMQ的原因是：

- **开发语言优势。** RocketMQ 使用 Java 语言开发，比起使用 Erlang 开发的 RabbitMQ 来说，有着更容易上手的阅读体验和受众。在遇到 RocketMQ 较为底层的问题时，大部分熟悉 Java 的同学都可以深入阅读其源码，分析、排查问题。
- **社区氛围活跃。** RocketMQ 是阿里巴巴开源且内部在大量使用的消息队列，说明 RocketMQ 的确是经得起残酷的生产环境考验的，并且能够针对线上环境复杂的需求场景提供相应的解决方案。
- **特性丰富。** 根据 RocketMQ 官方文档的列举，其高级特性达到了 **12 种**，例如顺序消息、事务消息、消息过滤、定时消息等。顺序消息、事务消息、消息过滤、定时消息。RocketMQ 丰富的特性，能够为我们在复杂的业务场景下尽可能多地提供思路及解决方案。

RocketMQ和Kafka的区别是什么？如何做技术选型？

Kafka的优缺点：

- 优点：首先，Kafka的最大优势就在于它的高吞吐量，在普通机器4CPU8G的配置下，一台机器可以抗住十几万的QPS，这一点还是相当优越的。Kafka支持集群部署，如果部分机器宕机不可用，则不影响Kafka的正常使用。
- 缺点：Kafka有可能会造成数据丢失，因为它在收到消息的时候，并不是直接写到物理磁盘的，而是先写入到磁盘缓冲区里面的。Kafka功能比较的单一 主要的就是支持收发消息，高级功能基本没有，就会造成适用场景受限。

RocketMQ是阿里巴巴开源的消息中间件，优缺点

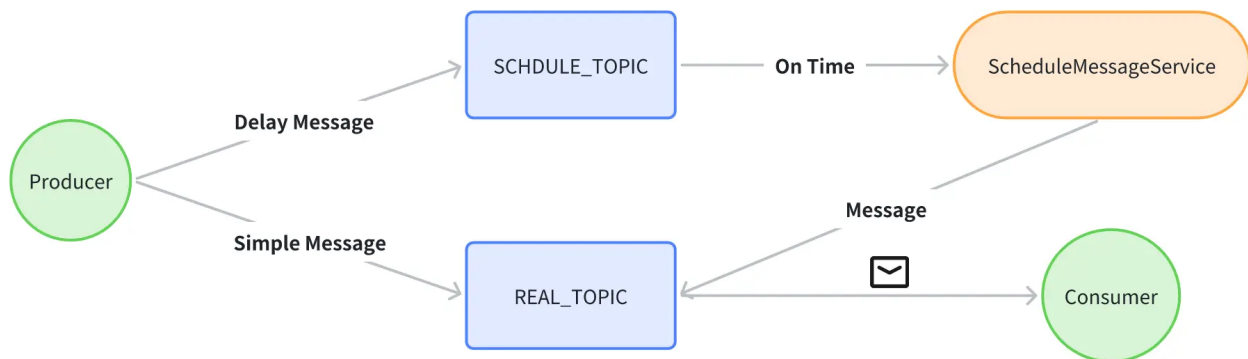
- 优点：支持功能比较多，比如延迟队列、消息事务等等，吞吐量也高，单机吞吐量达到 10 万级，支持大规模集群部署，线性扩展方便，Java语言开发，满足了国内绝大部分公司技术栈
- 缺点：性能相比 kafka 是弱一点，因为 kafka 用到了 sendfile 的零拷贝技术，而 RocketMQ 主要是用 mmap+write 来实现零拷贝。

该怎么选择呢？

- 如果我们业务只是收发消息这种单一类型的需求，而且可以允许小部分数据丢失的可能性，但是又要求极高的吞吐量和高性能的话，就直接选Kafka就行了，就好比我们公司想要收集和传输用户行为日志以及其他相关日志的处理，就选用的Kafka中间件。
- 如果公司的需要通过 mq 来实现一些业务需求，比如延迟队列、消息事务等，公司技术栈主要是Java语言的话，就直接一步到位选择RocketMQ，这样会省很多事情。

RocketMQ延时消息的底层原理

总体的原理示意图，如下所示：



broker 在接收到延时消息的时候，会将延时消息存入到延时Topic的队列中，然后ScheduleMessageService中，每个 queue 对应的定时任务会不停地被执行，检查 queue 中哪些消息已到设定时间，然后转发到消息的原始 Topic，这些消息就会被各自的 producer 消费了。

RocketMQ怎么处理分布式事务？

RocketMQ是一种最终一致性的分布式事务，就是说它保证的是消息最终一致性，而不是像2PC、3PC、TCC那样强一致分布式事务

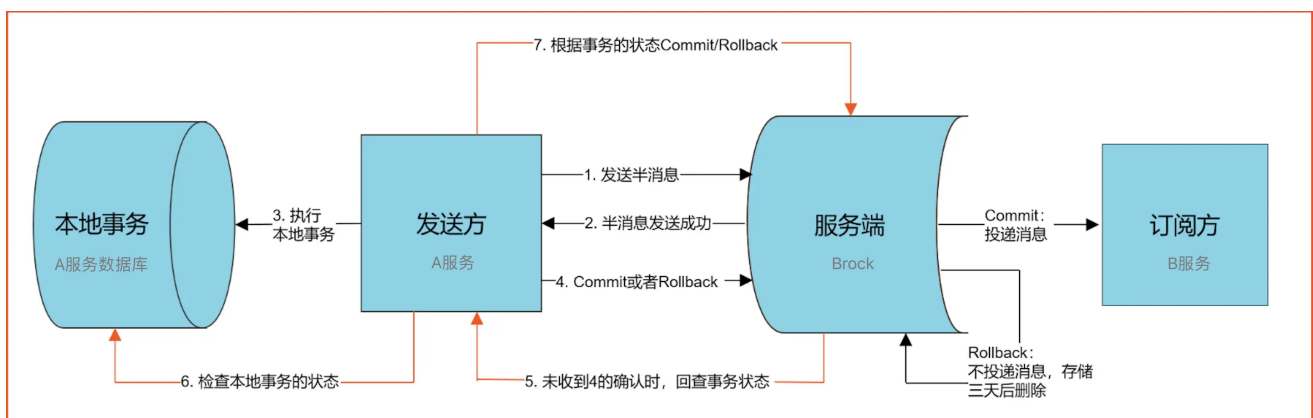
假设 **A** 给 **B** 转 **100块钱**，同时它们不是同一个服务上，现在目标就是 **A** 减100块钱，**B** 加100块钱。

实际情况可能有四种：

- 1) 就是A账户减100（成功），B账户加100（成功）
- 2) 就是A账户减100（失败），B账户加100（失败）
- 3) 就是A账户减100（成功），B账户加100（失败）
- 4) 就是A账户减100（失败），B账户加100（成功）

这里 **第1和第2** 种情况是能够保证事务的一致性的，但是 **第3和第4** 是无法保证事务的一致性的。

那 we 来看看下RocketMQ是如何来保证事务的一致性的。



分布式事务的流程如上图：

- 1、A服务先发送个Half Message（是指暂不能被Consumer消费的消息。Producer 已经把消息成功发送到了Broker 端，但此消息被标记为暂不能投递状态，处于该种状态下的消息称为半消息。需要 Producer对消息的二次确认后，Consumer才能去消费它）给Broker端，消息中携带 B服务 即将要+100元的信息。
- 2、当A服务知道Half Message发送成功后，那么开始第3步执行本地事务。
- 3、执行本地事务(会有三种情况1、执行成功。2、执行失败。3、网络等原因导致没有响应)
- 4.1)、如果本地事务成功，那么Product像Brock服务器发送Commit,这样B服务就可以消费该message。

- 4.2)、如果本地事务失败，那么Product像Brock服务器发送Rollback,那么就会直接删除上面这条半消息。
- 4.3)、如果因为网络等原因迟迟没有返回失败还是成功，那么会执行RocketMQ的回调接口，来进行事务的复查。

从上面流程可以得知 只有A服务本地事务执行成功，B服务才能消费该message。

那么 A账户减100（成功），B账户加100（失败），这时候B服务失败怎么办？

如果B最终执行失败，几乎可以断定就是代码有问题所以才引起的异常，因为消费端RocketMQ有重试机制，如果不是代码问题一般重试几次就能成功。

如果是代码的原因引起多次重试失败后，也没有关系，将该异常记录下来，由人工处理，人工兜底处理后，就可以让事务达到最终的一致性。

RocketMQ消息顺序怎么保证？

消息的有序性是指消息的消费顺序能够严格保存与消息的发送顺序一致。例如，一个订单产生了3条消息，分别是订单创建、订单付款和订单完成。在消息消费时，同一条订单要严格按照这个顺序进行消费，否则业务会发生混乱。同时，不同订单之间的消息又是可以并发消费的，比如可以先执行第三个订单的付款，再执行第二个订单的创建。

RocketMQ采用了局部顺序一致性的机制，实现了单个队列中的消息严格有序。也就是说，如果想要保证顺序消费，必须将一组消息发送到同一个队列中，然后再由消费者进行注意消费。

RocketMQ推荐的顺序消费解决方案是：安装业务划分不同的队列，然后将需要顺序消费的消息发往同一队列中即可，不同业务之间的消息仍采用并发消费。这种方式在满足顺序消费的同时提高了消息的处理速度，在一定程度上避免了消息堆积问题

RocketMQ 顺序消息的原理是：

- 在 Producer（生产者）把一批需要保证顺序的消息发送到同一个 MessageQueue
- Consumer（消费者）则通过加锁的机制来保证消息消费的顺序性，Broker 端通过对 MessageQueue 进行加锁，保证同一个 MessageQueue 只能被同一个 Consumer 进行消费。

RocketMQ怎么保证消息不被重复消费

在业务逻辑中实现**幂等性**，确保即使消息被重复消费，也不会影响业务状态。例如，对于支付或转账类操作，可以使用唯一订单号或事务ID作为幂等性的标识符，确保同样的操作只会被执行一次。

RocketMQ消息积压了，怎么办？

导致消息积压突然增加，最粗粒度的原因，只有两种：要么是发送变快了，要么是消费变慢了。

要解决积压的问题，可以通过**扩容消费端的实例数来提升总体的消费能力**。

如果短时间内没有足够的服务器资源进行扩容，没办法的办法是，将**系统降级**，通过关闭一些不重要的业务，减少发送方发送的数据量，最低限度让系统还能正常运转，服务一些重要业务。

kafka

对Kafka有什么了解吗？

Kafka特点如下：

- 高吞吐量、低延迟：kafka每秒可以处理几十万条消息，它的延迟最低只有几毫秒，每个topic可以分多个partition, consumer group 对partition进行consume操作。
- 可扩展性：kafka集群支持热扩展
- 持久性、可靠性：消息被持久化到本地磁盘，并且支持数据备份防止数据丢失
- 容错性：允许集群中节点失败（若副本数量为n,则允许n-1个节点失败）
- 高并发：支持数千个客户端同时读写

Kafka 为什么这么快？

- **顺序写入优化**：Kafka将消息顺序写入磁盘，减少了磁盘的寻道时间。这种方式比随机写入更高效，因为磁盘读写头在顺序写入时只需移动一次。
- **批量处理技术**：Kafka支持批量发送消息，这意味着生产者在发送消息时可以等待直到有足够的消息积累到一定量，然后再发送。这种方法减少了网络开销和磁盘I/O操作的次数，从而提高了吞吐量。
- **零拷贝技术**：Kafka使用零拷贝技术，可以直接将数据从磁盘发送到网络套接字，避免了在用户空间和内核空间之间的多次数据拷贝。这大幅降低了CPU和内存的负载，提高了数据传输效率。
- **压缩技术**：Kafka支持对消息进行压缩，这不仅减少了网络传输的数据量，还提高了整体的吞吐量。

kafka的模型介绍一下，kafka是推送还是拉取？

消费者模型

消息由生产者发送到kafka集群后，会被消费者消费。一般来说我们的消费模型有两种：推送模型(push)和拉取模型(pull)。

推送模型 (push)

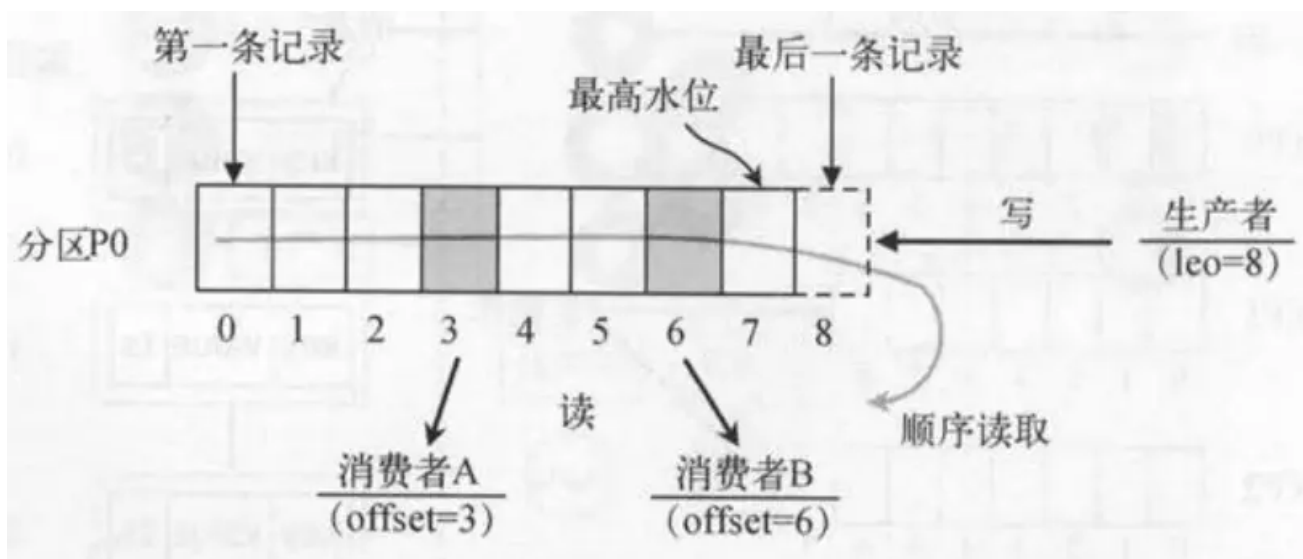
- 基于推送模型 (push) 的消息系统，有消息代理记录消费者的消费状态。
- 消息代理在将消息推送到消费者后，标记这条消息已经消费，但这种方式无法很好地保证消费被处理。
- 如果要保证消息被处理，消息代理发送完消息后，要设置状态为“已发送”，只要收到消费者的确认请求后才更新为“已消费”，这就需要代理中记录所有的消费状态，但显然这种方式不可取。

缺点：

- **push模式很难适应消费速率不同的消费者**
- 因为消息发送速率是由broker决定的，push模式的目标是尽可能以最快速度传递消息，但是这样很容易造成consumer来不及处理消息，典型的表现就是拒绝服务以及网络拥塞。

拉取模型(pull)

kafka采用拉取模型，由消费者自己记录消费状态，每个消费者互相独立地顺序拉取每个分区的信息。

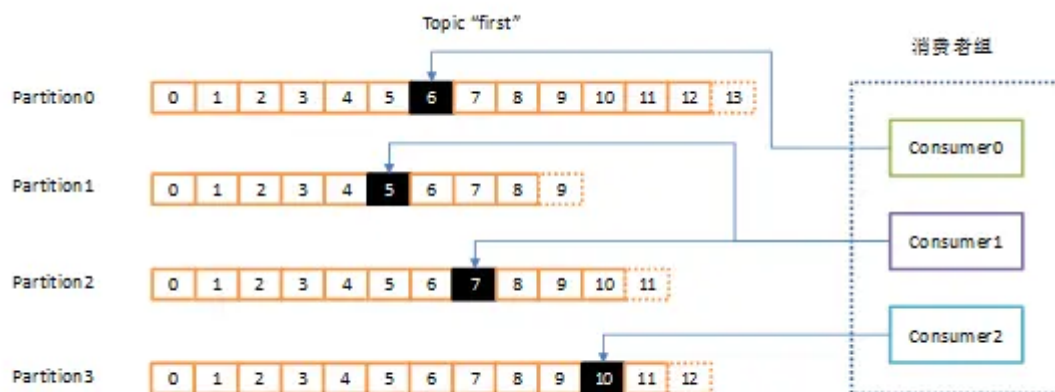


说明：

- 有两个消费者（不同消费者组）拉取同一个主题的消息，消费者A的消费进度是3，消费者B的消费进度是6。
- 消费者拉取的最大上限通过最高水位（watermark）控制，生产者最新写入的消息如果还没有达到备份数量，对消费者是不可见的。
- 这种由消费者控制偏移量的优点是：**消费者可以按照任意的顺序消费消息**。比如，消费者可以重置到旧的偏移量，重新处理之前已经消费过的消息；或者直接跳到最近的位置，从当前的时刻开始消费。

消费者组

kafka 消费者是以consumer group消费者组的方式工作，由一个或者多个消费者组成一个组，共同消费一个topic。每个分区在同一时间只能由group中的一个消费者读取，但是多个group可以同时消费这个partition。



上图中，有一个由三个消费者组成的group，有一个消费者读取主题中的两个分区，另外两个分别读取一个分区。某个消费者读取某个分区，也可以叫做某个消费者是某个分区的拥有者。

优点在于：

- 消费者可以通过水平扩展的方式同时读取大量的消息。
- 如果一个消费者失败了，那么其他的group成员会自动负载均衡读取之前失败的消费者读取的分区。

消费方式

kafka 消费者采用 pull（拉）模式从 broker中读取数据。

pull 的优点：

- pull 模式可以根据 consumer 的消费能力以适当的速率消费消息

缺点：

- 如果 kafka 没有数据，消费者可能会陷入循环中，一直返回空数据。针对这一点，Kafka 的消费者在消费数据时会传入一个时长参数 timeout，如果当前没有数据可供消费，consumer 会等待一段时间之后再返回，这段时长即为 timeout。

Kafka 如何保证顺序读取消息？

Kafka 可以保证在同一个分区内消息是有序的，生产者写入到同一分区的消息会按照写入顺序追加到分区日志文件中，消费者从分区中读取消息时也会按照这个顺序。这是 Kafka 天然具备的特性。

要在 Kafka 中保证顺序读取消息，需要结合生产者、消费者的配置以及合适的业务处理逻辑来实现。以下具体说明如何实现顺序读取消息：

- 生产者端确保消息顺序：为了保证消息写入同一分区从而确保顺序性，生产者需要将消息发送到指定分区。可以通过自定义分区器来实现，通过为消息指定相同的Key，保证相同Key的消息发送到同一分区。
- 消费者端保证顺序消费：消费者在消费消息时，需要单线程消费同一分区的消息，这样才能保证按顺序处理消息。如果使用多线程消费同一分区，就无法保证消息处理的顺序性。

Kafka 本身不能保证跨分区的消息顺序性，如果需要全局的消息顺序性，通常有以下两种方法：

- 只使用一个分区：将所有消息都写入到同一个分区，消费者也只从这个分区消费消息。但这种方式会导致 Kafka 的并行处理能力下降，因为 Kafka 的性能优势在于多分区并行处理。
- 业务层面保证：在业务代码中对消息进行编号或添加时间戳等标识，消费者在消费消息后，根据这些标识对消息进行排序处理。但这种方式会增加业务代码的复杂度。

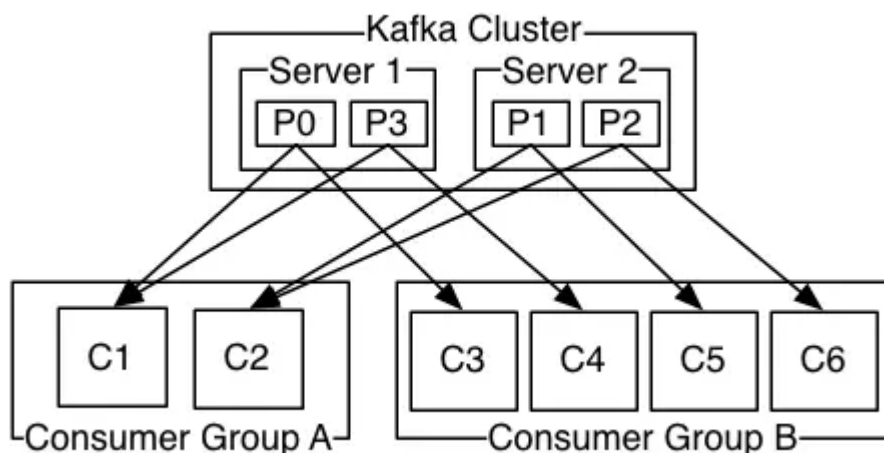
kafka 消息积压怎么办？

Kafka 消息积压是一个常见的问题，它可能会导致数据处理延迟，甚至影响业务的正常运行，下面是一些解决 Kafka 消息积压问题的常用方法：

- 增加消费者实例可以提高消息的消费速度，从而缓解决积压问题。你需要确保消费者组中的消费者数量不超过分区数量，因为一个分区同一时间只能被一个消费者消费。
- 增加 Kafka 主题的分区数量可以提高消息的并行处理能力。在创建新分区后，你需要重新平衡消费者组，让更多的消费者可以同时消费消息。

Kafka为什么一个分区只能由消费者组的一个消费者消费？这样设计的意义是什么？

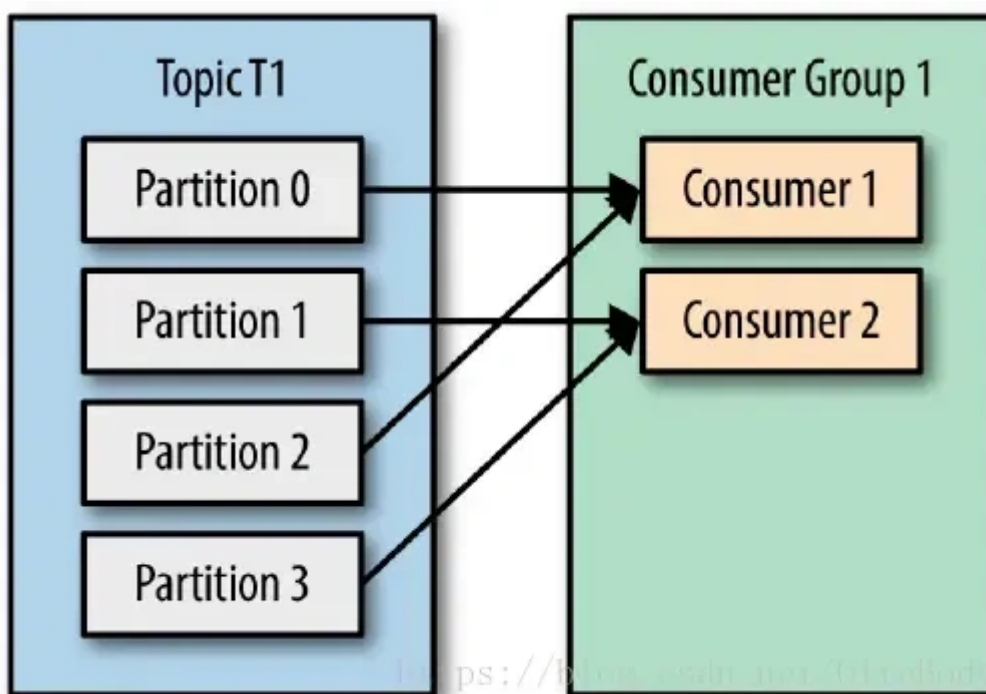
同一时刻，一条消息只能被组中的一个消费者实例消费



如果两个消费者负责同一个分区，那么就意味着两个消费者同时读取分区的信息，由于消费者自己可以控制读取消息的offset，就有可能C1才读到2，而C1读到1，C1还没处理完，C2已经读到3了，则会造成很多浪费，因为这就相当于多线程读取同一个消息，会造成消息处理的重复，且不能保证消息的顺序。

如果有一个消费主题topic，有一个消费组group，topic有10个分区，消费线程数和分区数的关系是怎么样的？

topic下的一个分区只能被同一个consumer group下的一个consumer线程来消费，但反之并不成立，即一个consumer线程可以消费多个分区的数据，比如Kafka提供的ConsoleConsumer，默认就只是一个线程来消费所有分区的数据。



所以，分区数决定了同组消费者个数的上限。

如果你的分区数是N，那么最好线程数也保持为N，这样通常能够达到最大的吞吐量。超过N的配置只是浪费系统资源，因为多出的线程不会被分配到任何分区。

消息中间件如何做到高可用？

消息中间件如何保证高可用呢？单机是没有高可用可言的，高可用都是对集群来说的，一起看下kafka的高可用吧。

Kafka 的基础集群架构，由多个 `broker` 组成，每个 `broker` 都是一个节点。当你创建一个 `topic` 时，它可以划分为多个 `partition`，而每个 `partition` 放一部分数据，分别存在于不同的 `broker` 上。也就是说，一个 `topic` 的数据，是分散放在多个机器上的，每个机器就放一部分数据。

有些伙伴可能有疑问，每个 `partition` 放一部分数据，如果对应的 `broker` 挂了，那这部分数据是不是就丢失了？那还谈什么高可用呢？

Kafka 0.8 之后，提供了复制副本机制来保证高可用，即每个 `partition` 的数据都会同步到其它机器上，形成多个副本。然后所有的副本会选举一个 `leader` 出来，让 `leader` 去跟生产和消费者打交道，其他副本都是 `follower`。写数据时，`leader` 负责把数据同步给所有的 `follower`，读消息时，直接读 `leader` 上的数据即可。如何保证高可用的？就是假设某个 `broker` 宕机，这个 `broker` 上的 `partition` 在其他机器上都有副本的。如果挂的是 `leader` 的 `broker` 呢？其他 `follower` 会重新选一个 `leader` 出来。

Kafka 和 RocketMQ 消息确认机制有什么不同？

Kafka 的消息确认机制有三种：0，1，-1：

- **ACK=0**：这是最不可靠的模式。生产者在发送消息后不会等待来自服务器的确认。这意味着消息可能会在发送之后丢失，而生产者将无法知道它是否成功到达服务器。
- **ACK=1**：这是默认模式，也是一种折衷方式。在这种模式下，生产者会在消息发送后等待来自分区领导者（`leader`）的确认，但不会等待所有副本（`replicas`）的确认。这意味着只要消息被写入分区领导者，生产者就会收到确认。如果分区领导者成功写入消息，但在同步到所有副本之前宕机，消息可能会丢失。
- **ACK=-1**：这是最可靠的模式。在这种模式下，生产者会在消息发送后等待所有副本的确认。只有在所有副本都成功写入消息后，生产者才会收到确认。这确保了消息的可靠性，但会导致更长的延迟。

RocketMQ 提供了三种消息发送方式：同步发送、异步发送和单向发送：

- **同步发送**：是指消息发送方发出一条消息后，会在收到服务端同步响应之后才发下一条消息的通讯方式。应用场景非常广泛，例如重要通知邮件、报名短信通知、营销短信系统等。
- **异步发送**：是指发送方发出一条消息后，不等服务端返回响应，接着发送下一条消息的通讯方式，但是需要实现异步发送回调接口（`SendCallback`）。消息发送方在发送了一条消息后，不需要等待服务端响应即可发送第二条消息。发送方通过回调接口接收服务端响应，并处理响应结果。适用于链路耗时较长，对响应时间较为敏感的业务场景，例如，视频上传后通知启动转码服务，转码完成后通知推送转码结果等。
- **单向发送**：发送方只负责发送消息，不等待服务端返回响应且没有回调函数触发，即只发送请求不等待应答。此方式发送消息的过程耗时非常短，一般在微秒级别。适用于某些耗时非常短，但对可靠性要求并不高的场景，例如日志收集。

Kafka 和 RocketMQ 的 broker 架构有什么区别

- Kafka 的 `broker` 架构：Kafka 的 `broker` 架构采用了分布式的设计，**每个 Kafka broker 是一个独立的服务实例**，负责存储和处理一部分消息数据。Kafka 的 `topic` 被分区存储在不同的 `broker` 上，实现了水平扩展和高可用性。
- RocketMQ 的 `broker` 架构：RocketMQ 的 `broker` 架构也是分布式的，但是**每个 RocketMQ broker 有主从之分**，一个主节点和多个从节点组成一个 `broker` 集群。主节点负责消息的写入和消费者的拉取，从节点负责消息的复制和消费者的负载均衡，提高了消息的可靠性和可用性。

RabbitMQ

RabbitMQ的特性你知道哪些？

RabbitMQ 以 **可靠性**、**灵活性** 和 **易扩展性** 为核心优势，适合需要稳定消息传递的复杂系统。其丰富的插件和协议支持使其在微服务、IoT、金融等领域广泛应用，比较核心的特性有如下：

- **持久化机制**：RabbitMQ 支持消息、队列和交换器的持久化。当启用持久化时，消息会被写入磁盘，即使 RabbitMQ 服务器重启，消息也不会丢失。例如，在声明队列时可以设置 `durable` 参数为 `true` 来实现队列的持久化：

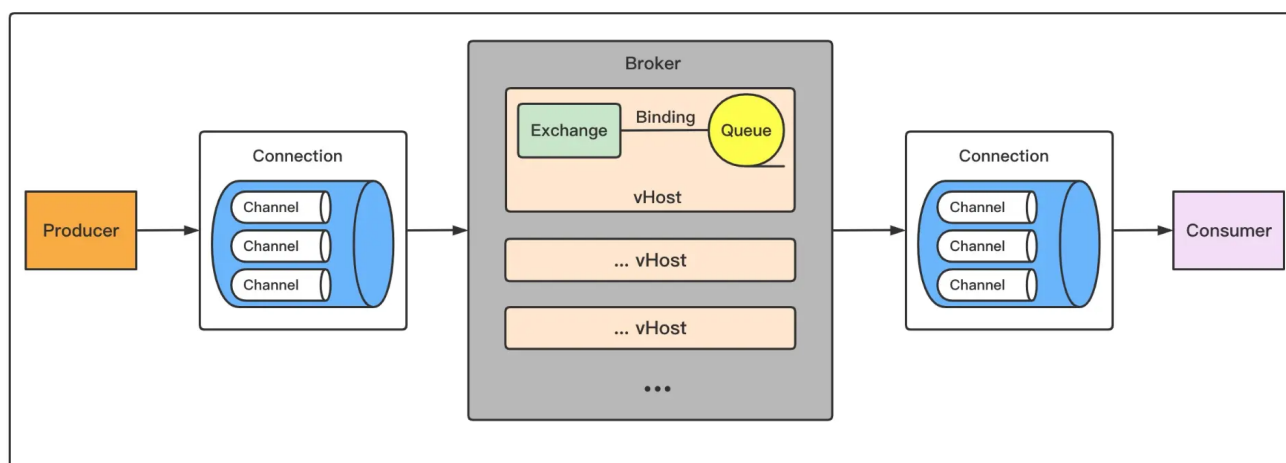
```
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()

# 声明一个持久化队列
channel.queue_declare(queue='durable_queue', durable=True)
```

- **消息确认机制**：提供了生产者确认和消费者确认机制。生产者可以设置 `confirm` 模式，当消息成功到达 RabbitMQ 服务器时，会收到确认消息；消费者在处理完消息后，可以向 RabbitMQ 发送确认信号，告知服务器该消息已被成功处理，服务器才会将消息从队列中删除。
- **镜像队列**：支持创建镜像队列，将队列的内容复制到多个节点上，提高消息的可用性和可靠性。当一个节点出现故障时，其他节点仍然可以提供服务，确保消息不会丢失。
- **多种交换器类型**：RabbitMQ 提供了多种类型的交换器，如直连交换器（Direct Exchange）、扇形交换器（Fanout Exchange）、主题交换器（Topic Exchange）和头部交换器（Headers Exchange）。不同类型的交换器根据不同的规则将消息路由到队列中。例如，扇形交换器会将接收到的消息广播到所有绑定的队列中；主题交换器则根据消息的路由键和绑定键的匹配规则进行路由。

RabbitMQ的底层架构是什么？



以下是 RabbitMQ 的一些核心架构组件和特性：

- **核心组件**：生产者负责发送消息到 RabbitMQ、消费者负责从 RabbitMQ 接收并处理消息、RabbitMQ 本身负责存储和转发消息。
- **交换机**：交换机接收来自生产者的消息，并根据 routing key 和绑定规则将消息路由到一个或多个队列。
- **持久化**：RabbitMQ 支持消息的持久化，可以将消息保存在磁盘上，以确保在 RabbitMQ 重启后消息不丢失，队列也可以设置为持久化，以保证其结构在重启后不会丢失。

- **确认机制**：为了确保消息可靠送达，RabbitMQ 使用确认机制，费者在处理完消息后发送确认给 RabbitMQ，未确认的消息会重新入队。
- **高可用性**：RabbitMQ 提供了集群模式，可以将多个 RabbitMQ 实例组成一个集群，以提高可用性和负载均衡。通过镜像队列，可以在多个节点上复制同一队列的内容，以防止单点故障。

 面试鸭 mianshiya.com

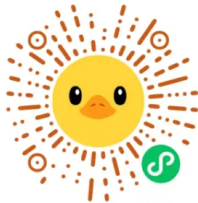
程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

刷高频题拿高薪offer



最新的图解文章都在公众号首发，别忘记关注哦！！如果你想加入百人技术交流群，扫码下方二维码回复「加群」。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群